Solution: Subsequence Pattern Matching

Problem Statement

Given a string and a pattern, write a method to count the number of times the pattern appears in the string as a subsequence.

Example 1: Input: string: "baxmx", pattern: "ax"
Output: 2
Explanation: {baxmx, baxmx}.

Example 2:

Input: string: "tomorrow", pattern: "tor"
Output: 4
Explanation: Following are the four occurences: {tomorrow, tomorrow, tomorrow, tomorrow}.

Basic Solution

This problem follows the `"Longest Common Subsequence" (LCS)` pattern and is quite similar to the `"Longest Repeating Subsequence"`; the difference is that we need to count the total occurrences of the subsequence.

A basic brute-force solution could be to try all the subsequences of the given string to count all that match the given pattern. We can match the pattern with the given string one character at a time, so we can do two things at any step:

1. If the pattern has a matching character with the string, we can recursively match for the remaining lengths of the pattern and the string.
2. At every step, we can always skip a character from the string to try to match the remaining string with the pattern. So we can start a recursive call by skipping one character from the string.

Our total count will be the sum of the counts returned by the above two options.

Here is the code:

Java

```java
class Solution {

  public int findSPMCount(String str, String pat) {
    return findSPMCountRecursive(str, pat, 0, 0);
  }

  private int findSPMCountRecursive(String str, String pat, int strIndex, int patIndex)

    // if we have reached the end of the pattern
    if(patIndex == pat.length())
      return 1;

    // if we have reached the end of the string but pattern has still some characters le
    if(strIndex == str.length())
      return 0;

    int c1 = 0;
    if(str.charAt(strIndex) == pat.charAt(patIndex))
      c1 = findSPMCountRecursive(str, pat, strIndex+1, patIndex+1);

    int c2 = findSPMCountRecursive(str, pat, strIndex+1, patIndex);
```

Run Code Result Stdout

You must run your code first

Execute

The time complexity of the above algorithm is exponential $O(2^m)$, where 'm' is the length of the string, as our recursion stack will not be deeper than `m`. The space complexity is $O(m)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems.

The two changing values to our recursive function are the two indexes `strIndex` and `patIndex`. Therefore, we can store the results of all the subproblems in a two-dimensional array. (Another alternative could be to use a hash-table whose key would be a string (strIndex + "|" + patIndex)).

Here is the code:

Java

```java
class Solution {

  public int findSPMCount(String str, String pat) {
    Integer[][] dp = new Integer[str.length()][pat.length()];
    return findSPMCountRecursive(dp, str, pat, 0, 0);
  }

  private int findSPMCountRecursive(Integer[][] dp, String str, String pat, int strIndex

    // if we have reached the end of the pattern
    if(patIndex == pat.length())
      return 1;

    // if we have reached the end of the string but pattern has still some characters le
    if(strIndex == str.length())
      return 0;

    if(dp[strIndex][patIndex] == null) {
      int c1 = 0;
      if(str.charAt(strIndex) == pat.charAt(patIndex))
        c1 = findSPMCountRecursive(dp, str, pat, strIndex+1, patIndex+1);
```

Run Code Result Stdout

You must run your code first

Execute

Bottom-up Dynamic Programming

Since we want to match all the subsequences of the given string, we can use a two-dimensional array to store our results. As mentioned above, we will be tracking separate indexes for the string and the pattern, so we will be doing two things for every value of `strIndex` and `patIndex`:

1. If the character at the `strIndex` (in the string) matches the character at `patIndex` (in the pattern), the count of the SPM would be equal to the count of SPM up to `strIndex–1` and `patIndex–1`.
2. At every step, we can always skip a character from the string to try matching the remaining string with the pattern; therefore, we can add the SPM count from the indexes `strIndex–1` and `patIndex`.

So our recursive formula would be:

```
if str[strIndex] == pat[patIndex] {
  dp[strIndex][patIndex] = dp[strIndex–1][patIndex–1]
}
dp[strIndex][patIndex] += dp[strIndex–1][patIndex]
```

Code

Here is the code for our bottom-up dynamic programming approach:

Java

```java

class Solution {

  public int findSPMCount(String str, String pat) {
    // every empty pattern has one match
    if(pat.length() == 0)
      return 1;

    if(str.length() == 0 || pat.length() > str.length())
      return 0;

    // dp[strIndex][patIndex] will be storing the count of SPM up to str[0..strIndex-1][
    int[][] dp = new int[str.length()+1][pat.length()+1];

    // for the empty pattern, we have one matching
    for(int i=0; i<=str.length(); i++)
      dp[i][0] = 1;

    for(int strIndex=1; strIndex<=str.length(); strIndex++) {
      for(int patIndex=1; patIndex<=pat.length(); patIndex++) {
        if(str.charAt(strIndex-1) == pat.charAt(patIndex-1))
```

Run Code Result Stdout

You must run your code first

Execute

The time and space complexity of the above algorithm is $O(m*n)$, where 'm' and 'n' are the lengths of the string and the pattern respectively.

← Previous
Subsequence Pattern Matching

Next →
Longest Bitonic Subsequence

☐ Mark as Completed